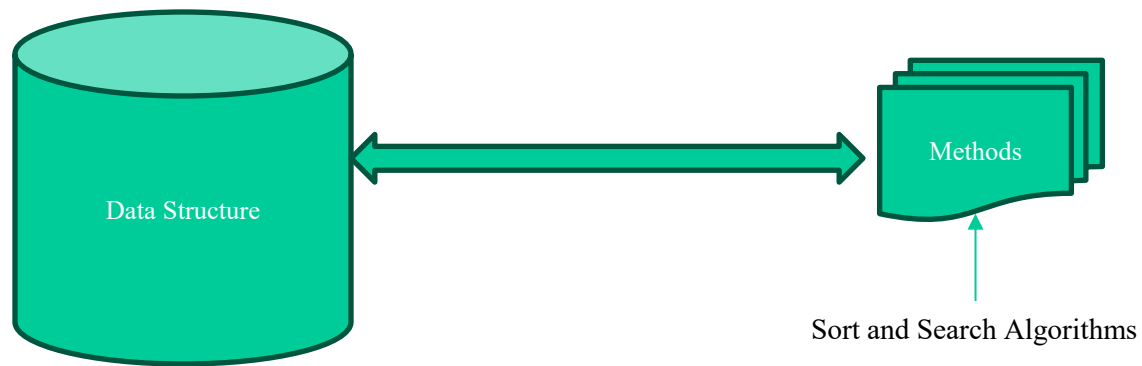


# Search & Sort Algorithms in Java



# Asymptotic notation

Amount of Time required to complete the process or to solve the problem.

- *Maximum time needed* to complete the process or to solve the problem.
- Determine how much time is needed for the *worst possible (Atmost)*

## Average / Expected-case

- Determine how much time is needed *on average* over all.

## Best Case

- Minimum time needed to complete the process or to solve the problem.
- Determine how much time is needed for best possible (Atleast)
- `int x[] = {34,21,45,76,90,23,48,81,62,70};`
- Looking for 34, 90, 62,70,101
- Best case of the algorithm  $O(1)$
- Average case  $O(n/2)$
- Worst case  $O(n)$

```
int sum=0;           1
for(int i=0; i<n; i++){  n
    sum= sum+i;       n
}
```

$f(x) = 1 + n + n;$   
 $= 1 + 2n;$

$O(n)$

```
int sum =0;          1
for(int i=0; i<n; i++){  n
for(int j=0; j<n; j++){  n
.....
}
```

$f(n) = 1 + nxn = 1 + n^2$

$O(n^2)$

```
int x[] = {1,2,3,4,5,6,7,8,9,10,11,12};
```

Search for 9

Compare  $x[0]$  with 9 (key)

$O(n)$  worst

$O(1)$  best case

$O(n/2)$  average case ,  $O(n)$

$O(n+1) = O(n)$ ,  $O(n/a) = O(n)$

Growth functions, 1,  $x$ ,  $x^2$ ,  $x^3$ ,  $\log n$ ,  $n \log n$ ,  $x^n$ ,

$$f(x) = x^2 + 3x + 4$$

Worst case

Best

Average

| X     | 0 | 1 | 2  | 3  | 4  | 5  | 6  |
|-------|---|---|----|----|----|----|----|
| F(x)  | 4 | 8 | 14 | 22 | 32 |    |    |
| $x^2$ | 0 | 1 | 4  | 9  | 16 | 25 | 36 |
| 3x    | 0 | 3 | 6  | 9  | 12 | 15 | 18 |
| 4     | 4 | 4 | 4  | 4  | 4  | 4  | 4  |

$$O(x^2)$$

$$O(4)$$

$$O(3x) = O(x)$$

# Search

- Necessary components needed to search an element in an array
  - Array containing the elements
  - Length of the Array
  - Key/Search Item for which you are searching
- Different Search Techniques are
  - **Linear Search / Sequential Search**
  - **Binary Search / Half Interval Search**

Both Techniques will return index of the specified value in the array

# Linear Search / Sequential Search

- **Linear search** is a method to find the index of a specified value in an array, that consists of checking all the elements, one at a time and in sequence, until the desired one is found.
- Linear search is the simplest search algorithm and it is a special case of **Brute-force search algorithm**.

# Searching an Array for a Specific Item

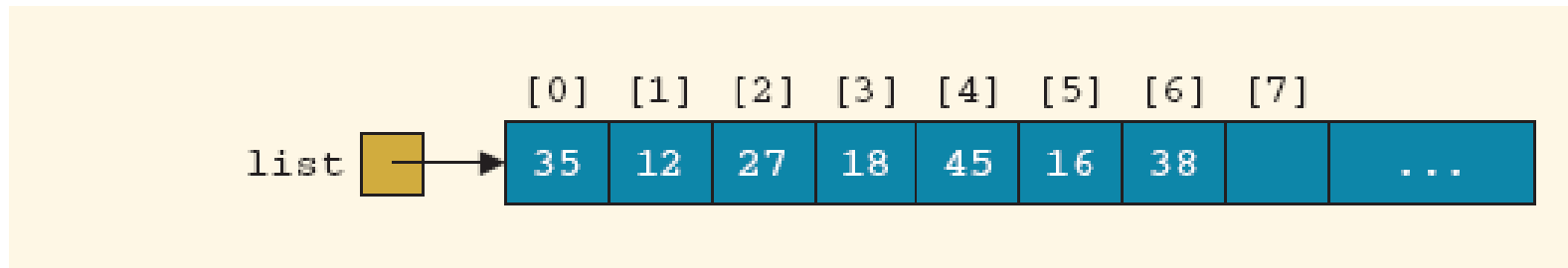


FIGURE 9-10 List of seven elements

## Example - 1

- Suppose that you want to determine whether 27 is in the array;
- First you compare 27 with `list[0]`
- Because `list[0] ≠ 27`, you then compare 27 with `list[1]`
- Because `list[1] ≠ 27`, you compare 27 with `list[2]`;  
Because `list[2] = 27`, the search stops
- This search is successful



# Searching an Array for a Specific Item

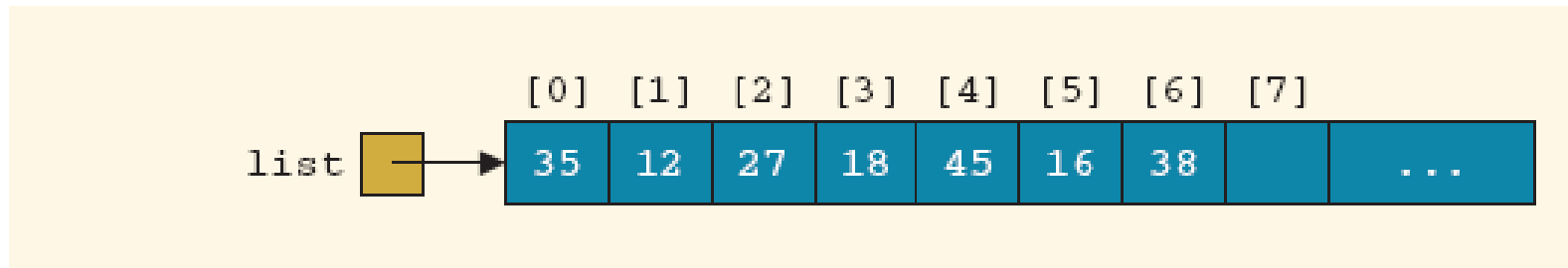


FIGURE 9-10 List of seven elements

- Search for 10
- Search starts at the first element in the array, that is, at `list[0]`
- This time, the search item, which is 10, is compared with every item in the array; eventually, no more data is left in the array to compare with the search item; this is an unsuccessful search

# Linear Search (continued)

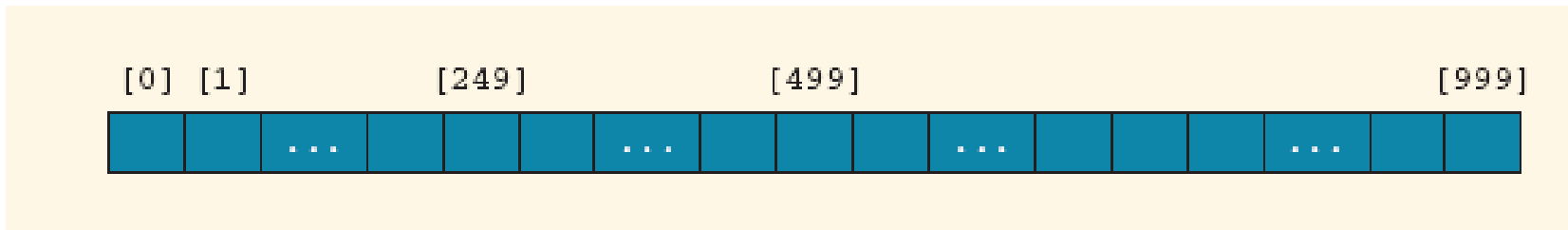


FIGURE 14-1 List of 1000 elements

## Example - 2

- If the search item is the second item in the array, the sequential search makes two **key comparisons** (also called **item comparisons**) to determine whether the search item is there in the array
- If the search item is the 900th item in the array, the sequential search makes 900 key comparisons to determine whether the search item is in the array
- If the search item is not in the array, the sequential search makes 1000 key comparisons

# Linear Search (continued)

- If `searchItem` is at the end of the array, it will take many comparisons to find `searchItem`
- If `searchItem` is not in the array, then we will compare `searchItem` with every element in the array
- A sequential search is therefore **not efficient for large arrays**; in fact, it can be proved that, on average, the number of comparisons made by a sequential search is equal to half the size of the array

# Linear Search / Sequential Search

## Best, Average and Worst case of Linear Search

- **Best case  $O(1)$**

Best case is when the value is equal to the first element of the array. In this case, only one comparison is needed.

- **Average Case**

On average, assuming the value searched for is in the array and each array element is equally likely to be the value searched for, the search visits only  $n/2$  elements.  $O(n/2) = O(n)$

- **Worst case**

This happens when the value being searched for is either the last element in the array, or is not in the array.

Therefore, if the array has more elements (No of elements  $> 8$ ), binary search will be faster and suitable.

*Asymptotic notation :*       $O(n)$

```
public int seqSearch(int[] array, int arrayLength, int searchItem)
{
    int loc;
    boolean found = false;
    loc = 0;
    while (loc < arrayLength)
    {
        if (array[loc] == searchItem)
        {
            found = true;
            break;
        }
        else
            loc++;
    }

    if (found==true)
        return loc;
    else
        return -1;
}
```

# OR

```
public int seqSearch(int array[], int Searchitem)
{
    boolean found = false;
    for (int i = 0; i < array.length; i++)
    {
        if (array[i] == Searchitem)
        {
            found = true;
            break;
        }
    }
    if (found==true)
        return i;
    else
        return -1;
}
```

## Class task

NB:

Based on what has been presented on linear search algorithm, create a function called `searchArray()`; that takes an array, array size and the search item as parameters. Use the following array to search for 43 from the list using the `searchArray` function.

Given array is

```
int []MayArray = {50,99,23,76,90,41};
```

NB: Use separate classes for the tester class and the main class which should be named `SearchAlgorithms`.

# Binary search / Half-interval search

- **Binary search** or **half-interval search** algorithm finds the index of a specified value (the input "key") within a **sorted array**.
- In each step, the algorithm compares the input key value with the key value of the middle element of the array. If the keys are matched, then its index is returned.
- Otherwise, if the search key is less than the middle element's key, then the algorithm repeats its action on the sub-array to the left of the middle element

OR

- If the search key is greater, then it will do on the sub-array to the right.
- If the remaining array to be searched is reduced to zero, even then the key cannot be found in the array, then a special "Not found" indication is returned.



|    |    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|----|
| 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  |
| 34 | 56 | 12 | 67 | 89 | 61 | 90 | 64 | 37 | 48 |
| 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  |
| 12 | 34 | 37 | 48 | 56 | 61 | 64 | 67 | 89 | 90 |

Given Key =64, identify low and high which is in this case low = 0 and high =9 so as to determine mid : $(mid = low + (high - low) / 2 )$

$mid = 0 + (9 + 0) / 2 = 4.5$  if( $x[mid] < key$ )  $low = mid + 1$

$mid = 4$ ,  $x[4] = 56 < 64$  therefore discard the lower part of the array starting from index 4.  $low = mid + 1$

$mid = low + (high - low) / 2 = 5 + (9 - 5) / 2 = 5 + 4 / 2 = 7$

$x[7] = 67 > 64$  therefore discard the upper part of the array including new mid if( $x[mid] > key$ )  $high = mid - 1$

$High = mid - 1 = 7 - 1 = 6$   $Mid = low + (high - low)$   
 $= 5 + (6 - 5) / 2 = 5 + 1 / 2 = 5.5 = 5$ ;  $x[5] = 61 < 64$

$Low = mid + 1 = 5 + 1 = 6$   $X[6] = 64$  found

# Binary Search Algorithm

- Search item is compared with middle element of array.
  - If search item = middle element, search is complete
  - If search item < middle element of array, search is restricted to first half of the array
  - If search item > middle element of array, search second half of the array

# Binary Search Algorithm (continued)

- Determine whether 75 is in the array

|      |     |     |     |     |     |     |     |     |     |     |      |      |
|------|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|------|------|
|      | [0] | [1] | [2] | [3] | [4] | [5] | [6] | [7] | [8] | [9] | [10] | [11] |
| list | 4   | 8   | 19  | 25  | 34  | 39  | 45  | 48  | 66  | 75  | 89   | 95   |

FIGURE 14-19 List of length 12

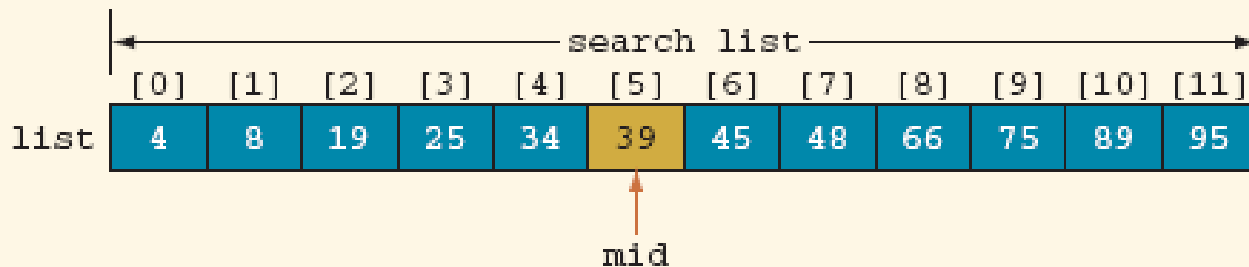


FIGURE 14-20 Search list, `list[0]...list[11]`

# Binary Search Algorithm (continued)

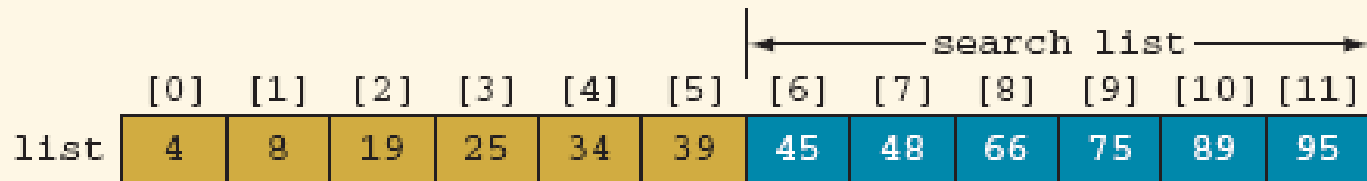


FIGURE 14-21 Search list, `list[6]...list[11]`

# Binary Search Algorithm (continued)

|      |     |     |     |     |     |     |     |     |     |     |      |      |
|------|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|------|------|
|      | [0] | [1] | [2] | [3] | [4] | [5] | [6] | [7] | [8] | [9] | [10] | [11] |
| list | 4   | 8   | 19  | 25  | 34  | 39  | 45  | 48  | 66  | 75  | 89   | 95   |

FIGURE 14-22 Sorted list for binary search

`searchItem = 89`

| Iteration | first | last | mid | list[mid] | Number of key comparisons |
|-----------|-------|------|-----|-----------|---------------------------|
| 1         | 0     | 11   | 5   | 39        | 2                         |
| 2         | 6     | 11   | 8   | 66        | 2                         |
| 3         | 9     | 11   | 10  | 89        | 1 (found is <b>true</b> ) |

`searchItem = 34`

| Iteration | first | last | mid | list[mid] | Number of key comparisons |
|-----------|-------|------|-----|-----------|---------------------------|
| 1         | 0     | 11   | 5   | 39        | 2                         |
| 2         | 0     | 4    | 2   | 19        | 2                         |
| 3         | 3     | 4    | 3   | 25        | 2                         |
| 4         | 4     | 4    | 4   | 34        | 1 (found is <b>true</b> ) |

# Binary Search Algorithm (continued)

|      |     |     |     |     |     |     |     |     |     |     |      |      |
|------|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|------|------|
|      | [0] | [1] | [2] | [3] | [4] | [5] | [6] | [7] | [8] | [9] | [10] | [11] |
| list | 4   | 8   | 19  | 25  | 34  | 39  | 45  | 48  | 66  | 75  | 89   | 95   |

FIGURE 10-34 Sorted list for binary search

`searchItem = 22`

| Iteration | <code>first</code> | <code>last</code> | <code>mid</code>                                                           | <code>list[mid]</code> | Number of key comparisons |
|-----------|--------------------|-------------------|----------------------------------------------------------------------------|------------------------|---------------------------|
| 1         | 0                  | 11                | 5                                                                          | 39                     | 2                         |
| 2         | 0                  | 4                 | 2                                                                          | 19                     | 2                         |
| 3         | 3                  | 4                 | 3                                                                          | 25                     | 2                         |
| 4         | 3                  | 2                 | the loop stops (because <code>first &gt; last</code> ) unsuccessful search |                        |                           |

# Binary search algorithm(conti..)

- Requires a sorted array before searching.
- A binary search halves the number of items to check with each iteration, so locating an item (or determining its absence) takes logarithmic time.
- A binary search is a special case of **Divide and conquer search algorithm**.
- **Best case:**
  - Best case is one time if the first data was happened to be the target data to be found.
  - Best case comes when search key appears at middle of array
  - *Asymptotic notation :*  $O(1)$
- **Average case:**
  - *Asymptotic notation :*  $O(\log n)$
- **Worst case:**
  - Means the longest time it takes to search array
  - Search key at the end of array, key at the beginning of array and key not in array
  - *Asymptotic notation :*  $O(\log n)$

$$1 < \text{sqr}(n) < \log_2 n < n < n \log_2 n < n^2 < 2^n < n^n$$



```
public int binarySearch(int[] array, int arrayLength,
                        int searchItem)
{
    int first = 0;
    int last = arrayLength - 1;
    int mid;
    boolean found = false;
    while (first <= last && !found)
    {
        mid = (first + last) / 2;
        if (array[mid] == searchItem)
            found = true;
        else if (array[mid] > searchItem)
            last = mid - 1;
        else
            first = mid + 1;
    }
    if (found)
        return mid;
    else
        return -1;
} //end binarySearch
```

```
public int binarySearch(int[] array, int arrayLength,
int searchItem)
{
    int first = 0;
    int last = arrayLength - 1;
    int mid;
    boolean found = false;
    while (first <= last)
    {
        mid = (first + last) / 2;
        if (array[mid] == searchItem)
        {
            found = true;
            break;
        }
        else if (array[mid] > searchItem)
            last = mid - 1;
        else
            first = mid + 1;
    }
    if (found)
        return mid;
    else
        return -1;
}
```

# When can you say that Linear Search and Binary search Algorithms are effective?

- *Linear Search is effective when the array is short and almost sorted and ineffective when the array is long and involves many swapping*
- *Binary Search is efficient when the array is long and sorted but inefficient when the array is short or unsorted*

# Differentiate between linear and binary search

| <i>Linear Search</i>                                                                                     | <i>Binary Search</i>                                                                           |
|----------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| It's a special case of brute-force search algorithm.                                                     | It's a special case of divide and conquer search algorithm.                                    |
| <i>Does not require a sorted array</i>                                                                   | <i>Requires a sorted array before searching</i>                                                |
| Finds the index by Searching each element one at a time and in sequence, until the desired one is found. | Finds the index by checking the search key with the middle element of the array.               |
| <i>Worst case: key at the end of array and key not in array</i>                                          | <i>Worst case: key at the end of array, key at the beginning of array and key not in array</i> |
| <i>Best case: key at beginning of array</i>                                                              | <i>Best case: key at middle of array</i>                                                       |